

Day 2 — PHP เชื่อม DB / API / Login

คอร์ส Backend เบื้องต้น: PHP + SQL (phpMyAdmin)

จัดทำโดย AtSoftMaker

Day 2 — PHP เชื่อมฐานข้อมูล / API / Login

วันที่สองเราจะให้ **PHP** มาเป็นตัวกลางคุยกับฐานข้อมูลที่สร้างไว้เมื่อวาน
แล้วต่อยอดเป็น **API** และ **ระบบ Login**

คุณเคยทำ Frontend มาแล้ว วันนี้คือการเข้าใจว่า "ฝั่งเซิร์ฟเวอร์" ทำงานอย่างไร และส่งข้อมูลให้ Frontend ได้อย่างไร

จุดประสงค์การเรียนรู้

เมื่อจบ Day 2 คุณจะสามารรถ:

- เขียน PHP พื้นฐาน (ตัวแปร, array, if, loop, function)
- เชื่อม PHP เข้ากับ MySQL ด้วย **mysqli** และแสดงข้อมูลออกหน้าเว็บ
- ทำหน้าเว็บ **CRUD** ครบ (แสดง/เพิ่ม/แก้/ลบ)
- สร้าง **API** ที่คืนข้อมูลเป็น **JSON**
- ทำระบบ **Login + Session** และรู้วิธียกระดับความปลอดภัย (prepared statement + hash)

เตรียมตัวก่อนเริ่ม

1. เปิด XAMPP → Start **Apache** และ **MySQL**
2. สร้างโฟลเดอร์โปรเจกต์ใน **htdocs** เช่น **htdocs/atsoft/**
3. คัดลอกไฟล์จากโฟลเดอร์ **src/** ไปไว้ใน **htdocs/atsoft/**
4. วันนี้เราจะใช้ฐานข้อมูลใหม่ชื่อ **atsoft_day2** — สร้างได้จากไฟล์ **src/setup.sql**
5. เปิดทดสอบผ่าน **http://localhost/atsoft/ไฟล์.php**

สารบัญ Day 2 (ทำตามลำดับ)

ลำดับ	ไฟล์	ประเภท	เนื้อหา
1	01_sheet_php_basic.md	ซีท	PHP พื้นฐาน + เชื่อม DB ด้วย mysqli
2	02_lab_php_crud.md	Lab 3	ทำหน้าเว็บ CRUD
3	03_sheet_api.md	ซีท	สร้าง API คืน JSON
4	04_lab_api.md	Lab 4	ทำ Record API
5	05_sheet_login.md	ซีท	Session + Login (+ hash แบบ optional)
6	06_lab_login.md	Lab 5	ทำระบบ Register/Login

ไฟล์ประกอบ:

- [src/](#) — โค้ดตัวอย่าง/เริ่มต้น (db.php, setup.sql, crud/, api/, auth/)
- [answers/](#) — เฉลยโค้ดเต็มแต่ละ Lab (+ เวอร์ชัน hash)

เริ่มที่ [01_sheet_php_basic.md](#)

ซีท 01 · PHP พื้นฐาน + เชื่อมต่อฐานข้อมูล

จุดประสงค์

เขียน PHP พื้นฐานได้ และใช้ PHP ดึงข้อมูลจาก MySQL มาแสดงบนหน้าเว็บ

1. PHP กับการรับ-ส่งข้อมูล (เรียก API คุยกับฐานข้อมูล)

PHP ทำงานฝั่ง **เซิร์ฟเวอร์ (Backend)** หน้าที่หลักของเราในคอร์สนี้คือให้ PHP เป็น "ตัวกลางของ API" — รับคำขอจากหน้าเว็บ แล้วไปคุยกับฐานข้อมูล MySQL แล้วส่งผลลัพธ์กลับไป

```
[ หน้าเว็บ / fetch ] --ส่ง request--> [ PHP (API) ] --query--> [ MySQL ]
      ^                               |
      |                               |
      |_____ ส่งผลกลับ (HTML/JSON) _____|
```

หน้าเว็บส่งข้อมูลเข้ามาให้ PHP ได้ 2 ช่องทางหลัก เราจะ เน้น **method="POST"** เป็นหลัก และใช้ **GET** เสริม:

ช่องทาง	ข้อมูลมากับ	เหมาะกับ
\$_POST (เน้น)	การ submit ฟอर्म method="POST" หรือ body ของ fetch	ส่งข้อมูล "เข้าไปทำงาน" เช่น เพิ่ม/แก้/ลบ/ ล็อกอิน
\$_GET	ค่าใน URL เช่น ?id=5	อ่าน/ค้นหา/ส่ง id ผ่านลิงก์

ทำไมเน้น POST: ค่าที่ส่งไม่โผล่ใน URL (ปลอดภัยกว่า) และเหมาะกับการ "เปลี่ยนแปลงข้อมูล" ในฐานข้อมูล ส่วน GET ใช้ตอน แค่ "อ่าน" หรือเปิดลิงก์

รับค่าที่ส่งมาแบบ **POST** (ใช้บ่อยสุด):

```
<?php
// ฟอर्मที่ตั้ง method="POST" จะส่งค่ามาที่ $_POST
$name = $_POST['name'] ?? '';
$price = $_POST['price'] ?? 0;
// เดี่ยวเอา $name, $price ไป INSERT ลงฐานข้อมูลด้วย mysqli (ดูข้อ 3)
?>
```

และรับค่าแบบ **GET** (ตอนอ่าน/ส่ง id):

```
<?php
$id = $_GET['id'] ?? null; // เช่นเปิด list.php?id=3
?>
```

ในคอร์สนี้เชื่อมฐานข้อมูลด้วยไทรเวอร์ **mysql** เท่านั้น (ไม่ใช่ PDO)

2. ไวยากรณ์พื้นฐาน

ไฟล์ PHP ลงท้ายด้วย `.php` และโค้ด PHP อยู่ในแท็ก `<?php ... ?>`

```
<?php
echo "สวัสดี Backend!"; // echo = พิมพ์ออกหน้าจอ (เหมือน console.log แต่ออกหน้าเว็บ)
?>
```

2.1 ตัวแปร (ขึ้นต้นด้วย \$)

```
<?php
$name = "สมชาย";
$age = 25;
$price = 19.50;
$isActive = true;

echo $name; // สมชาย
echo "อายุ " . $age; // เชื่อมข้อความด้วยจุด (.) → อายุ 25
?>
```

ต่างจาก JS: เชื่อมข้อความใช้จุด `.` ไม่ใช่เครื่องหมายบวก `+`

2.2 Array (อาเรย์)

```
<?php
// array แบบรายการ
$fruits = ["แอปเปิล", "กล้วย", "ส้ม"];
echo $fruits[0]; // แอปเปิล

// array แบบ key => value (เหมือน object ใน JS)
$person = ["name" => "สมชาย", "age" => 25];
echo $person["name"]; // สมชาย
?>
```

2.3 เงื่อนไข if

```
<?php
$score = 75;
if ($score >= 80) {
    echo "ดีมาก";
} elseif ($score >= 50) {
    echo "ผ่าน";
} else {
    echo "ไม่ผ่าน";
}
?>
```

2.4 วนซ้ำ (loop)

```
<?php
// for
for ($i = 1; $i <= 3; $i++) {
    echo "รอบที่ $i <br>";
}

// foreach (วนใน array)
$fruits = ["แอปเปิล", "กล้วย", "ส้ม"];
foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
?>
```

`<br>` คือแท็ก HTML ขึ้นบรรทัดใหม่ — เพราะผลลัพธ์ออกไปแสดงในเบราว์เซอร์

2.5 ฟังก์ชัน

```
<?php
function บวก($a, $b) {
    return $a + $b;
}
echo บวก(3, 5);    // 8
?>
```

2.6 แทรก PHP ใน HTML

```
<!DOCTYPE html>
<html>
<body>
  <h1>รายชื่อผลไม้</h1>
  <ul>
    <?php
      $fruits = ["แอปเปิล", "กล้วย", "ส้ม"];
      foreach ($fruits as $fruit) {
        echo "<li>$fruit</li>";
      }
    ?>
  </ul>
</body>
</html>
```

3. เชื่อมต่อฐานข้อมูลด้วย mysqli

เราจะใช้ **mysqli** แบบ procedural (เขียนเป็นฟังก์ชัน) เพราะเข้าใจง่ายสำหรับผู้เริ่มต้น

3.1 ขั้นตอน 4 ข้อ

1. เชื่อมต่อ (connect)
2. ส่งคำสั่ง SQL (query)
3. อ่านผลลัพธ์ (fetch)
4. (ปิดการเชื่อมต่อ — PHP ปิดให้เองเมื่อจบสคริปต์)

3.2 ไฟล์เชื่อมต่อกลาง db.php

เราเตรียมไว้ให้แล้วใน `src/db.php` :

```
<?php
$host = 'localhost';
$user = 'root';
$pass = ''; // XAMPP รหัส root เป็นค่าว่าง
$dbname = 'atsoft_day2';

$conn = mysqli_connect($host, $user, $pass, $dbname);
if (!$conn) {
    die('เชื่อมต่อไม่สำเร็จ: ' . mysqli_connect_error());
}
mysqli_set_charset($conn, 'utf8mb4'); // รองรับภาษาไทย
```

3.3 อ่านข้อมูล (SELECT) แล้วแสดงผล

สร้างไฟล์ `list.php` :

```
<?php
require 'db.php'; // ได้ตัวแปร $conn

$sql = "SELECT * FROM products";
$result = mysqli_query($conn, $sql); // ส่งคำสั่ง

echo "<h1>รายการสินค้า</h1>";
echo "<table border='1' cellpadding='8'>";
echo "<tr><th>ID</th><th>ชื่อ</th><th>ราคา</th><th>สต็อก</th></tr>";

// อ่านทีละแถว
while ($row = mysqli_fetch_assoc($result)) {
    echo "<tr>";
    echo "<td>" . $row['id'] . "</td>";
    echo "<td>" . $row['name'] . "</td>";
    echo "<td>" . $row['price'] . "</td>";
    echo "<td>" . $row['stock'] . "</td>";
    echo "</tr>";
}
echo "</table>";
?>
```

เปิด <http://localhost/atsoft/list.php> จะเห็นข้อมูลจากฐานข้อมูลออกมาเป็นตาราง

อธิบายฟังก์ชันสำคัญ:

| ฟังก์ชัน | หน้าที่ |

| :-- | :-- |

| `mysqli_connect(...)` | เชื่อมต่อฐานข้อมูล คืนค่า "connection" |

| `mysqli_query($conn, $sql)` | ส่งคำสั่ง SQL ไปรัน |

| `mysqli_fetch_assoc($result)` | ดึงผลลัพธ์ออกมาทีละแถว เป็น array (`$row['ชื่อคอลัมน์']`) |

3.4 เพิ่มข้อมูล (INSERT) จาก PHP

```
<?php
require 'db.php';

$name = "ปากกาแดง";
$category = "อุปกรณ์";
$price = 12.00;
$stock = 50;

$sql = "INSERT INTO products (name, category, price, stock)
VALUES ('$name', '$category', $price, $stock)";

if (mysqli_query($conn, $sql)) {
    echo "เพิ่มสินค้าสำเร็จ!";
} else {
    echo "ผิดพลาด: " . mysqli_error($conn);
}
?>
```

หมายเหตุความปลอดภัย: การเอาตัวแปรมาต่อใน SQL ตรงๆ แบบนี้ เสี่ยงต่อการโจมตี SQL Injection เราเขียนแบบนี้ก่อนเพื่อให้เข้าใจง่าย แล้วจะเรียน **prepared statement** ที่ปลอดภัยกว่าในซีท Login (ข้อ 05)

4. รับค่าจากฟอร์ม (GET / POST)

HTML ฟอร์มส่งข้อมูลมาให้ PHP ผ่าน 2 ช่องทาง:

- `$_GET` — ค่ามากับ URL (เช่น `?id=5`) เหมาะกับการค้นหา/ลิงก์
- `$_POST` — ค่ามากับการ submit ฟอร์ม เหมาะกับการเพิ่ม/แก้ไขข้อมูล

```
<!-- ฟอร์ม -->
<form method="POST" action="save.php">
  ชื่อ: <input type="text" name="name">
  <button type="submit">บันทึก</button>
</form>
```

```
<!-- save.php -->
<?php
$name = $_POST['name']; // รับค่าที่กรอกมา
echo "ได้รับชื่อ: " . $name;
?>
```

รับค่าจาก URL `list.php?id=3` ด้วย `$_GET['id']`

5. สรุป

- PHP รันฝั่งเซิร์ฟเวอร์ สร้าง HTML ส่งให้เบราว์เซอร์
- เชื่อม DB ด้วย `mysqli_connect` → `mysqli_query` → `mysqli_fetch_assoc`
- ใช้ `require 'db.php'` เพื่อไม่ต้องเขียนโค้ดเชื่อมต่อซ้ำ
- รับค่าจากผู้ใช้ด้วย `$_GET` / `$_POST`

ไปต่อ: [02_lab_php_crud.md](#)

Lab 3 · ทำหน้าเว็บ CRUD ด้วย PHP

เป้าหมาย

สร้างหน้าเว็บที่ แสดง / เพิ่ม / แก้ไข / ลบ ข้อมูลสินค้าได้ครบวงจร โดยใช้ PHP + mysqli

สิ่งที่ต้องเตรียม

1. เปิด XAMPP → Start Apache + MySQL
2. สร้างฐานข้อมูล: phpMyAdmin → แท็บ SQL → วางเนื้อหา `src/setup.sql` → Go
3. คัดลอกไฟล์เตอร์ทั้งหมดใน `src/` ไปไว้ที่ `htdocs/atsoft/`
 - โครงสร้างควรเป็น: `htdocs/atsoft/db.php` และ `htdocs/atsoft/crud/index.php` ...
4. ทดสอบเปิด `http://localhost/atsoft/crud/index.php`

ในไฟล์เตอร์ `src/crud/` มีโค้ดตัวอย่างที่ทำงานได้ครบให้แล้ว

โจทย์ Lab นี้: อ่านโค้ดให้เข้าใจทีละไฟล์ แล้วลองดัดแปลง/เพิ่มฟีเจอร์ตามข้อด้านล่าง

ส่วนที่ 1: ทำความเข้าใจโค้ด (อ่าน + ลองรัน)

เปิดอ่านและรันทีละไฟล์ แล้วตอบคำถามในใจ:

ไฟล์	ทำหน้าที่ CRUD ตัวไหน	คำถามชวนคิด
<code>index.php</code>	Read (แสดงรายการ)	<code>mysqli_fetch_assoc</code> ทำงานยังไงใน while loop?
<code>create.php</code>	Create (เพิ่ม)	ฟอร์มส่งข้อมูลด้วย method อะไร?
<code>edit.php</code>	Update (แก้ไข)	ดึง id มาจากไหน? เติมค่าเดิมในฟอร์มยังไง?
<code>delete.php</code>	Delete (ลบ)	ทำไม delete ใช้ <code>\$_GET['id']</code> ?

ข้อ 1.1 เปิด `index.php` แล้วลองเพิ่มสินค้าผ่านปุ่ม "เพิ่มสินค้า" → แก้ไข → ลบ ให้ครบทุกปุ่ม

ข้อ 1.2 เปิดดูในฐานข้อมูล (phpMyAdmin → Browse) ว่าข้อมูลเปลี่ยนตามจริงไหม

ส่วนที่ 2: ดัดแปลงโค้ด

ข้อ 2.1 ในหน้า `index.php` เพิ่มคอลัมน์ "มูลค่ารวม" = `price × stock` ของแต่ละสินค้า

คำนวณใน PHP: `<?= $row['price'] * $row['stock'] ?>`

ข้อ 2.2 ทำให้สินค้าที่ **สต็อกน้อยกว่า 30** แสดงพื้นหลังแถวเป็นสีแดงอ่อน (เตือนของใกล้หมด)

ใน `<tr>` ใส่เงื่อนไข: `<tr style="<?= $row['stock'] < 30 ? 'background:#ffe0e0' : '' ?>">`

ข้อ 2.3 ใน `create.php` เพิ่มการตรวจสอบ: ถ้าราคาติดลบ ไม่ให้บันทึก และแสดงข้อความเตือน

ตรวจก่อน INSERT: `if ($price < 0) { $error = 'ราคาต้องไม่ติดลบ'; } else { ... }`

ส่วนที่ 3: สร้างเองตั้งแต่ต้น (ท้าทาย)

ข้อ 3.1 สร้างหน้าใหม่ `report.php` ที่แสดงสรุป:

- จำนวนสินค้าทั้งหมด (`COUNT(*)`)
- มูลค่ารวมของสต็อกทั้งหมด (`SUM(price * stock)`)
- จำนวนสินค้าแยกตามหมวด (`GROUP BY category`)

ใช้ SQL จาก Day 1 + แสดงผลด้วย PHP เหมือน `index.php`

เช็คความเข้าใจ

- [] เข้าใจว่าไฟล์ไหนทำ C/R/U/D ตัวใด
- [] ดัดแปลงการแสดงผลใน `index.php` ได้
- [] เพิ่มการตรวจสอบข้อมูลก่อนบันทึกได้
- [] เขียนหน้าสรุป (report) ด้วย Aggregate ได้

เฉลย: เฉลย Lab 3

ไปต่อ: [03_sheet_api.md](#)

ซีท 03 · สร้าง API คินข้อมูลเป็น JSON

จุดประสงค์

เข้าใจว่า API คืออะไร และสร้าง API ด้วย PHP ที่ส่งข้อมูลกลับเป็น **JSON** ได้

1. API คืออะไร?

ตอนเรียน Frontend คุณอาจเคยใช้ `fetch()` ดึงข้อมูลจากที่ไหนสักแห่งมาแสดง — ที่ที่ส่งข้อมูลกลับมานั้นแหละคือ **API**

API = ช่องทางให้โปรแกรมคุยกับโปรแกรม โดยรับคำขอ แล้วส่ง **ข้อมูล** กลับ (ไม่ใช่หน้า HTML สวยๆ แต่เป็นข้อมูลดิบให้โปรแกรมอื่นเอาไปใช้)

เปรียบเทียบ:

- หน้าเว็บ `index.php` → ส่ง **HTML** ให้คนดู
- API `api.php` → ส่ง **JSON** ให้โปรแกรม (เช่น JavaScript, มือถือ) เอาไปใช้ต่อ

ตัวอย่างข้อมูล JSON ที่ API ส่งกลับ:

```
{
  "success": true,
  "data": [
    { "id": 1, "name": "น้ำดื่ม 600ml", "price": "7.00" },
    { "id": 2, "name": "น้ำโซดา 325ml", "price": "10.00" }
  ]
}
```

2. ทำให้ PHP ส่ง JSON

2 ขั้นตอนสำคัญ:

1. บอกเบราว์เซอร์ว่ากำลังส่ง JSON ด้วย `header()`
2. แปลง array ของ PHP เป็นข้อความ JSON ด้วย `json_encode()`

```
<?php
require 'db.php';

header('Content-Type: application/json; charset=utf-8'); // บอกว่านี่คือ JSON

$result = mysqli_query($conn, "SELECT * FROM products");

$products = [];
while ($row = mysqli_fetch_assoc($result)) {
    $products[] = $row; // เก็บแต่ละแถวเข้า array
}

// แปลง array → JSON แล้วพิมพ์ออกไป
echo json_encode([
    "success" => true,
    "data" => $products
], JSON_UNESCAPED_UNICODE); // UNESCAPED_UNICODE = ให้ภาษาไทยอ่านออก ไม่กลายเป็น \u...
?>
```

เปิด <http://localhost/atsoft/api.php> จะเห็นข้อมูลเป็น JSON ล้วนๆ

`JSON_UNESCAPED_UNICODE` สำคัญมากสำหรับภาษาไทย ไม่งั้นจะได้ น้ำ แทน "น้ำ"

3. รับพารามิเตอร์ (ให้ API ฉลาดขึ้น)

3.1 รับค่าจาก URL ด้วย \$_GET

```
// api.php?id=2 → ค้นสินค้าตัวเดียว
$id = $_GET['id'] ?? null;
if ($id) {
    $result = mysqli_query($conn, "SELECT * FROM products WHERE id = $id");
    $product = mysqli_fetch_assoc($result);
    echo json_encode(["success" => true, "data" => $product], JSON_UNESCAPED_UNICODE);
}
```

3.2 รู้ว่าเป็น method อะไร (GET/POST/PUT/DELETE)

API ที่ดีจะแยกการทำงานตาม HTTP method:

Method	ใช้ทำ	ตรงกับ CRUD
GET	อ่านข้อมูล	Read
POST	เพิ่มข้อมูล	Create
PUT	แก้ข้อมูล	Update
DELETE	ลบข้อมูล	Delete

```
$method = $_SERVER['REQUEST_METHOD']; // ได้ "GET", "POST", ...

if ($method === 'GET') {
    // อ่านข้อมูล
} elseif ($method === 'POST') {
    // เพิ่มข้อมูล
}
```

3.3 รับข้อมูล JSON ที่ส่งเข้ามา (ตอน POST/PUT)

เวลา Frontend ส่ง JSON มา (เช่น `fetch` ด้วย `body: JSON.stringify(...)`) ฝั่ง PHP อ่านแบบนี้:

```
$input = json_decode(file_get_contents('php://input'), true);
// $input จะเป็น array เช่น $input['name'], $input['price']
```

4. โครงสร้าง Response ที่ดี

ออกแบบให้ตอบกลับรูปแบบเดียวกันเสมอ จะได้ใช้ง่าย:

```
// สำเร็จ
echo json_encode(["success" => true, "data" => $data], JSON_UNESCAPED_UNICODE);

// ผิดพลาด
http_response_code(400); // ส่งรหัสสถานะ (400 = คำขอผิด, 404 = ไม่พบ, 500 = เซิร์ฟเวอร์พัง)
echo json_encode(["success" => false, "message" => "ไม่พบข้อมูล"], JSON_UNESCAPED_UNICODE);
```

5. ทดสอบ API อย่างไร

- **GET** ง่ายสุด: เปิด URL ในเบราว์เซอร์ได้เลย
- **POST/PUT/DELETE**: ใช้เครื่องมือทดสอบ เช่น
- **Postman** (โปรแกรมยอดนิยม)
- หรือเขียน `fetch()` ใน Console ของเบราว์เซอร์:

```
js fetch('http://localhost/atsoft/api/record_api.php', { method: 'POST', headers: {
  'Content-Type': 'application/json' }, body: JSON.stringify({ name: 'สินค้าใหม่', category:
  'ทดสอบ', price: 10, stock: 5 }) }).then(r => r.json()).then(console.log);
```

จุดเชื่อมกับ Frontend: นี่คือการทำให้หน้าเว็บที่คุณเคยทำ "ดึงข้อมูลจริง" ได้!

6. สรุป

- API ส่ง **ข้อมูล (JSON)** ไม่ใช่หน้า HTML

- ใช้ `header('Content-Type: application/json') + json_encode(..., JSON_UNESCAPED_UNICODE)`
- แยกการทำงานตาม `$_SERVER['REQUEST_METHOD']`
- รับ JSON เข้ามาด้วย `json_decode(file_get_contents('php://input'), true)`
- ตอบกลับรูปแบบเดียวกันเสมอ: `{success, data}` หรือ `{success, message}`

ไปต่อ: [04_lab_api.md](#)

Lab 4 · สร้าง Record API

เป้าหมาย

สร้างและทดสอบ API ที่จัดการข้อมูลสินค้าครบ CRUD ผ่าน HTTP method

สิ่งที่ต้องเตรียม

1. ฐานข้อมูล atsoft_day2 พร้อมตาราง products (จาก `src/setup.sql`)
2. คัดลอกไฟล์ `src/api/record_api.php` ไปไว้ที่ `htdocs/atsoft/api/`
3. (แนะนำ) ติดตั้ง **Postman** ไว้ทดสอบ POST/PUT/DELETE

ส่วนที่ 1: ทดสอบ API ที่ให้มา

ข้อ 1.1 (GET ทั้งหมด) เปิดในเบราว์เซอร์:

```
http://localhost/atsoft/api/record_api.php
```

ควรเห็น JSON รายการสินค้าทั้งหมด `{ "success": true, "data": [...] }`

ข้อ 1.2 (GET ตัวเดียว) เปิด:

```
http://localhost/atsoft/api/record_api.php?id=1
```

ควรเห็นสินค้า id=1 ตัวเดียว

ข้อ 1.3 (POST เพิ่มสินค้า) ใช้ Console ของเบราว์เซอร์ (กด F12 → Console) วาง:

```
fetch('http://localhost/atsoft/api/record_api.php', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ name: 'เจลล้างมือ', category: 'อุปกรณ์', price: 39.0, stock: 80 })  
}).then(r => r.json()).then(console.log);
```

ควรได้ `{ success: true, id: ... }` แล้วลอง GET ทั้งหมดดูว่ามีของใหม่

ข้อ 1.4 (DELETE) ลบสินค้าที่เพิ่งเพิ่ม (แทน `<id>` ด้วยเลขจริง):

```
fetch('http://localhost/atsoft/api/record_api.php?id=<id>', { method: 'DELETE' })  
  .then(r => r.json()).then(console.log);
```

ส่วนที่ 2: อ่านเข้าใจโค้ด

ตอบในใจ:

- `switch ($method)` ทำหน้าที่อะไร?
- ทำไม POST ใช้ `json_decode(file_get_contents('php://input'), true)` ?
- `(int)$id` และ `mysqli_real_escape_string(...)` มีไว้ทำไม? (คำใบ้: ความปลอดภัย)
- ฟังก์ชัน `respond()` ช่วยให้โค้ดสั้นลงยังไง?

ส่วนที่ 3: ต่อยอด

ข้อ 3.1 เพิ่มความสามารถค้นหา: `GET ...?search=น้ำ` ให้คืนเฉพาะสินค้าที่ชื่อมีคำค้น

ในเคส GET เพิ่มเงื่อนไข: ถ้ามี `$_GET['search']` ให้ใส่ `WHERE name LIKE '%...%'`

ข้อ 3.2 เพิ่ม endpoint สรุป: `GET ...?summary=1` คืน `{ total_products, total_value }`

ใช้ `COUNT(*)` และ `SUM(price*stock)`

ข้อ 3.3 ทำให้ POST ตรวจสอบว่า `price` และ `stock` ต้องไม่ติดลบ ถ้าติดลบให้ตอบ 400 พร้อมข้อความ

โจทย์ท้าทาย

ข้อ 4.1 สร้างหน้า HTML + JavaScript (`shop.html`) ที่ `fetch` ข้อมูลจาก API นีมาแสดงเป็นการ์ดสินค้า

นี่คือการเชื่อม Frontend (ที่คุณถนัด) เข้ากับ Backend (ที่เพิ่งเรียน) เข้าด้วยกัน!

เช็คความเข้าใจ

- [] เรียก API ด้วย GET/POST/DELETE ได้
- [] เข้าใจการแยกงานตาม HTTP method
- [] รับ-ส่งข้อมูลเป็น JSON ได้
- [] ต่อยอดเพิ่ม endpoint ค้นหา/สรุปได้

เฉลย: เฉลย Lab 4

ไปต่อ: [05_sheet_login.md](#)

ซีท 05 · ระบบ Login + Session

จุดประสงค์

ทำาระบบสมัครสมาชิก + เข้าสู่ระบบ + จำสถานะผู้ใช้ (Session) + ป้องกันหน้าที่ต้องล็อกอิน
แล้วเรียนรู้วิธี ยกระดับความปลอดภัย (prepared statement + hash) เป็นขั้นตอนเสริม

เราจะทำเป็น 2 เวอร์ชัน:

1. เวอร์ชันเข้าใจง่าย — เก็บรหัสผ่านแบบ plain text (เพื่อให้เห็น flow ชัดๆ ก่อน)
2. เวอร์ชันปลอดภัย (Optional) — เพิ่ม hash + prepared statement (ทำเมื่อเข้าใจ flow แล้ว)

1. Session คืออะไร?

HTTP เป็นโปรโตคอลที่ "ขี้ลืม" — แต่ครั้งที่เปิดหน้าเว็บ เซิร์ฟเวอร์ไม่รู้ว่เราเป็นใคร
Session คือวิธีที่เซิร์ฟเวอร์ "จำ" ผู้ใช้ไว้ระหว่างที่ยังใช้งานอยู่ (เช่น จำว่าล็อกอินแล้ว)

วิธีใช้ใน PHP:

```
<?php
session_start(); // เปิด session (ต้องเรียกบนสุดของไฟล์ ก่อน echo ใดๆ)
$_SESSION['user'] = 'somchai'; // เก็บข้อมูลลง session
echo $_SESSION['user']; // อ่านกลับมาได้ในหน้าอื่นๆ
session_destroy(); // ล้าง session (ตอน logout)
?>
```

2. ภาพรวม Flow ของระบบ Login

```
[สมัครสมาชิก register] → บันทึก username + password ลงตาราง users
[เข้าสู่ระบบ login] → ตรวจสอบว่า username/password ตรงกับในตารางไหม
├─ ตรง → เก็บลง $_SESSION แล้วพาไปหน้า dashboard
└─ ไม่ตรง → แจ้ง "รหัสผ่านไม่ถูกต้อง"
[หน้าที่ต้องล็อกอิน] → เช็คว่ามี $_SESSION หรือยัง ถ้าไม่มีดึงไป login
[ออกจากระบบ logout] → ล้าง session
```

เราใช้ตาราง `users` (สร้างไว้แล้วใน `setup.sql`):

```
CREATE TABLE users (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  username    VARCHAR(50) NOT NULL UNIQUE,  
  password    VARCHAR(255) NOT NULL,  
  full_name   VARCHAR(100),  
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

3. เวอร์ชันเข้าใจง่าย (plain text ก่อน)

3.1 สมัครสมาชิก register.php

```
<?php  
require '../db.php';  
  
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
  $username = $_POST['username'];  
  $password = $_POST['password']; // ยังเก็บแบบดิบ (จะแก้ทีหลัง)  
  $fullname = $_POST['full_name'];  
  
  $sql = "INSERT INTO users (username, password, full_name)  
        VALUES ('$username', '$password', '$fullname')";  
  
  if (mysqli_query($conn, $sql)) {  
    echo "สมัครสำเร็จ! <a href='login.php'>ไปเข้าสู่ระบบ</a>";  
  } else {  
    echo "ผิดพลาด: " . mysqli_error($conn);  
  }  
}  
?  
>  
<form method="POST">  
  Username: <input name="username" required><br>  
  Password: <input type="password" name="password" required><br>  
  ชื่อ-สกุล: <input name="full_name"><br>  
  <button>สมัครสมาชิก</button>  
</form>
```

3.2 เข้าสู่ระบบ login.php

```
<?php
session_start();
require '../db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    $sql = "SELECT * FROM users WHERE username = '$username'";
    $result = mysqli_query($conn, $sql);
    $user = mysqli_fetch_assoc($result);

    if ($user && $user['password'] === $password) { // ตรวจสอบรหัสผ่าน
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['username'] = $user['username'];
        header('Location: dashboard.php');
        exit;
    } else {
        $error = "username หรือ password ไม่ถูกต้อง";
    }
}

?>
<?php if (!empty($error)) echo "<p style='color:red'>$error</p>"; ?>
<form method="POST">
    Username: <input name="username" required><br>
    Password: <input type="password" name="password" required><br>
    <button>เข้าสู่ระบบ</button>
</form>
<a href="register.php">ยังไม่มีบัญชี? สมัครสมาชิก</a>
```

3.3 หน้าที่ต้องล็อกอิน dashboard.php

```
<?php
session_start();

// ด้านป้องกัน: ถ้ายังไม่ล็อกอิน เด้งกลับไป login
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}

?>
<h1>ยินดีต้อนรับ <?= htmlspecialchars($_SESSION['username']) ?> </h1>
<p>นี่คือหน้าที่เห็นได้เฉพาะคนที่ล็อกอินแล้ว</p>
<a href="logout.php">ออกจากระบบ</a>
```

3.4 ออกจากระบบ logout.php

```
<?php
session_start();
session_destroy(); // ล้างข้อมูล session ทั้งหมด
header('Location: login.php');
exit;
```

แค่นี้ก็ได้ระบบ Login ที่ทำงานได้แล้ว! ลองสมัคร → เข้าสู่ระบบ → เข้า dashboard → ออกจากระบบ

4. เวอร์ชันปลอดภัย (Optional — ทำเมื่อเข้าใจ flow ข้างบนแล้ว)

เวอร์ชันข้างบนมี 2 ช่องโหว่ใหญ่ ในงานจริง:

1. เก็บรหัสผ่านแบบ plain text — ถ้าฐานข้อมูลรั่ว รหัสผ่านโดนเห็นหมด
2. SQL Injection — เอา `$_POST` ต่อใน SQL ตรงๆ ผู้ไม่หวังดีอาจป้อนคำสั่งอันตราย

4.1 ปัญหา SQL Injection คืออะไร

ถ้าผู้ใช้กรอก username เป็น:

```
' OR '1'='1
```

SQL จะกลายเป็น `... WHERE username = '' OR '1'='1'` ซึ่งเป็นจริงเสมอ → เข้าระบบได้โดยไม่ต้องรู้รหัส!

ทางแก้: Prepared Statement — แยก "คำสั่ง" ออกจาก "ข้อมูล" ฐานข้อมูลจะไม่ตีความข้อมูลเป็นคำสั่ง

```
// แทนการต่อสตริง ไข่ ? เป็นช่องเสียบค่า
$stmt = mysqli_prepare($conn, "SELECT * FROM users WHERE username = ?");
mysqli_stmt_bind_param($stmt, "s", $username); // "s" = string
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$user = mysqli_fetch_assoc($result);
```

4.2 การ Hash รหัสผ่าน

Hash = แปลงรหัสผ่านเป็นข้อความสุ่มที่ย้อนกลับไม่ได้ เก็บแต่ค่าที่ hash แล้ว

เวลาตรวจ ก็ hash รหัสที่ผู้ใช้กรอก แล้วเทียบกับที่เก็บไว้

ตัวอย่างด้วย SHA-256 (ตามที่โจทย์ระบุ):

```
// ตอนสมัคร: hash ก่อนเก็บ
$hashed = hash('sha256', $password);
// เก็บ $hashed ลงฐานข้อมูลแทน $password

// ตอนล็อกอิน: hash รหัสที่กรอก แล้วเทียบ
$inputHashed = hash('sha256', $password);
if ($user['password'] === $inputHashed) {
    // รหัสถูกต้อง
}
```

ทดสอบดูได้ว่า `hash('sha256', '1234')` จะได้ข้อความยาว 64 ตัวอักษรเสมอ และเหมือนเดิมทุกครั้ง

หมายเหตุสำคัญสำหรับงานจริง:

SHA-256 เร็วเกินไป จึงถูกเดา (brute-force) ได้ง่ายถ้าใช้กับรหัสผ่าน

ในระบบจริงควรใช้ `password_hash()` ของ PHP (ใช้ bcrypt + ใส่ salt อัตโนมัติ):

```
php $hashed = password_hash($password, PASSWORD_DEFAULT); // ตอนสมัคร if
```

```
(password_verify($password, $user['password'])) { ... } // ตอนตรวจ  
ในคอร์สนี้เราฝึก SHA-256 เพื่อ "เข้าใจหลักการ hash" ก่อน แล้วค่อยรู้ว่าของจริงใช้ password_hash()
```

5. สรุป

- `session_start()` + `$_SESSION` ใช้จำว่าใครล็อกอินอยู่
- Flow: register → login (เช็ครหัส) → เก็บ session → ป้องกันหน้า → logout
- เริ่มจากเวอร์ชันเข้าใจง่ายก่อน แล้วยกระดับด้วย **prepared statement** (กัน SQL Injection) และ **hash** (กันรหัสรั่ว)
- งานจริงใช้ `password_hash()` / `password_verify()`

ไปต่อ: [06_lab_login.md](#)

Lab 5 • ระบบ Register / Login

เป้าหมาย

ทำระบบสมาชิกครบวงจร: สมัคร → เข้าสู่ระบบ → หน้าที่ต้องล็อกอิน → ออกจากระบบ
แล้ว (ถ้าพร้อม) ยกระดับด้วย hash + prepared statement

สิ่งที่ต้องเตรียม

- ฐานข้อมูล atsoft_day2 มีตาราง users แล้ว (จาก `src/setup.sql`)
- คัดลอกไฟล์ใน `src/auth/` ไปไว้ที่ `htdocs/atsoft/auth/`
- เปิด <http://localhost/atsoft/auth/register.php>

ส่วนที่ 1: ทำให้ระบบทำงาน (เวอร์ชันเข้าใจง่าย)

ข้อ 1.1 สมัครสมาชิก 1 บัญชี (เช่น username test , password 1234)

ข้อ 1.2 ไปที่ phpMyAdmin → ตาราง users → Browse

สังเกต: รหัสผ่านถูกเก็บเป็น 1234 ตรงๆ — นี่คือนิสัยที่ไม่ปลอดภัย ที่เราจะแก้ในส่วนที่ 3

ข้อ 1.3 เข้าสู่ระบบด้วยบัญชีที่สมัคร → ควรเข้าหน้า dashboard.php ได้

ข้อ 1.4 ทดสอบความปลอดภัยของ session:

- ขณะยังไม่ล็อกอิน ลองเปิด dashboard.php ตรงๆ → ควรถูกดึงไป login.php
- ล็อกอินแล้วกด "ออกจากระบบ" → แล้วลองเปิด dashboard.php อีกครั้ง → ควรถูกดึงออกอีก

ส่วนที่ 2: เข้าใจ Flow (ตอบในใจ)

- `session_start()` ทำไมต้องอยู่บรรทัดบนสุด (ก่อน HTML)?
- `$_SESSION['user_id']` ถูกตั้งค่าตอนไหน และถูกเช็คที่ไหน?
- `session_destroy()` ทำอะไร?
- ทำไม dashboard.php ต้องมี "ด่านป้องกัน" ตอนต้นไฟล์?

ส่วนที่ 3: ยกระดับความปลอดภัย (Optional แต่แนะนำมาก)

ทำส่วนนี้เมื่อเข้าใจ flow ส่วนที่ 1 แล้ว

ข้อ 3.1 (Hash รหัสผ่าน) แก้ register.php ให้ hash รหัสก่อนเก็บ:

```
$hashed = hash('sha256', $password);  
// แล้วเก็บ $hashed แทน $password
```

และแก้ login.php ให้ hash รหัสที่กรอกก่อนเทียบ:

```
$inputHashed = hash('sha256', $password);  
if ($user && $user['password'] === $inputHashed) { ... }
```

บัญชีเก่า (plain) จะล็อกอินไม่ได้แล้ว — ให้สมัครใหม่ หรือ `TRUNCATE TABLE users;` ก่อน

```
// register.php (ตอนบันทึกข้อมูล)  
$hashed = hash('sha256', $password);  
$sql = "INSERT INTO users (username, password, full_name)  
VALUES ('$username', '$hashed', '$fullname')";  
  
// login.php (ตอนตรวจสอบรหัสผ่าน)  
$inputHashed = hash('sha256', $password);  
$sql = "SELECT * FROM users WHERE username = '$username'";  
$result = mysqli_query($conn, $sql);  
$user = mysqli_fetch_assoc($result);  
if ($user && $user['password'] === $inputHashed) {  
    $_SESSION['user_id'] = $user['id'];  
    $_SESSION['username'] = $user['username'];  
    header('Location: dashboard.php');  
    exit;  
}
```

ข้อ 3.2 (ตรวจผลใน DB) สมัครบัญชีใหม่ แล้วดูในตาราง users อีกครั้ง

ตอนนี้รหัสผ่านควรเป็นข้อความสุ่มยาว 64 ตัวอักษร อ่านไม่ออกแล้ว

ข้อ 3.3 (กัน SQL Injection) เปลี่ยน login.php ให้ใช้ prepared statement:

```
$stmt = mysqli_prepare($conn, "SELECT * FROM users WHERE username = ?");  
mysqli_stmt_bind_param($stmt, "s", $username);  
mysqli_stmt_execute($stmt);  
$result = mysqli_stmt_get_result($stmt);  
$user = mysqli_fetch_assoc($result);
```

```
// login.php ฉบับสมบูรณ์ที่ใช้ Prepared Statement และ Hash ป้องกัน SQL Injection
$inputHashed = hash('sha256', $password);

$stmt = mysqli_prepare($conn, "SELECT * FROM users WHERE username = ?");
mysqli_stmt_bind_param($stmt, "s", $username);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$user = mysqli_fetch_assoc($result);

if ($user && hash_equals($user['password'], $inputHashed)) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['username'] = $user['username'];
    header('Location: dashboard.php');
    exit;
} else {
    $error = "username หรือ password ไม่ถูกต้อง";
}
```

ข้อ 3.4 (ทดสอบช่องโหว่) ก่อน แก้เป็น prepared statement ลองกรอก username:

```
' OR '1'='1
```

สังเกตว่าเวอร์ชันเก่าอาจมีพฤติกรรมผิดปกติ แต่เวอร์ชัน prepared statement จะปลอดภัย

โจทย์ท้าทาย

ข้อ 4.1 เพิ่มคอลัมน์ role (เช่น admin / user) ในตาราง users แล้วทำให้ dashboard แสดงเมนูต่างกันตาม role

```
ALTER TABLE users ADD role VARCHAR(20) DEFAULT 'user'; แล้วเก็บ role ลง session ด้วย
```

```
// 1. เพิ่มคอลัมน์ในฐานข้อมูล (รัน SQL ใน phpMyAdmin)
// ALTER TABLE users ADD role VARCHAR(20) NOT NULL DEFAULT 'user';

// 2. ใน login.php (ตอนเช็คผ่านแล้ว) ให้เก็บ role ลง session
$_SESSION['role'] = $user['role'];

// 3. ใน dashboard.php (แสดงเมนูและข้อมูลตามระดับสิทธิ์)
if ($_SESSION['role'] === 'admin') {
    echo '<a href="manage_users.php">⚙️ จัดการผู้ใช้ (เฉพาะแอดมิน)</a>';
} else {
    echo '<p>คุณเป็นผู้ใช้ทั่วไป</p>';
}
```

เช็คความเข้าใจ

- [] ทำระบบ register/login/logout ที่ทำงานได้
- [] เข้าใจการใช้ session ป้องกันหน้า
- [] hash รหัสผ่านด้วย SHA-256 ได้ และเข้าใจว่าทำไมต้อง hash

- [] ใช้ prepared statement กัน SQL Injection ได้
- [] รู้ว่างานจริงควรใช้ `password_hash()`

เฉลย: เฉลย Lab 5 · เวอร์ชันปลดล็อกเพิ่มเติม: [answers/auth_secure/](#)

จบ Day 2! ไปต่อ: [Day 3 — Project](#)

เฉลย Lab 3 · หน้าเว็บ CRUD

โค้ดเต็มที่ทำงานได้อยู่ในโฟลเดอร์ `../src/crud/` แล้ว — ส่วนนี้เฉลยเฉพาะข้อที่ให้ดัดแปลง

ส่วนที่ 2: ดัดแปลงโค้ด

2.1 เพิ่มคอลัมน์ "มูลค่ารวม" ใน `index.php`

เพิ่มหัวตาราง และในแต่ละแถว:

```
<!-- ในแถวหัวตาราง <tr> เพิ่ม -->
<th>มูลค่ารวม</th>

<!-- ในแต่ละแถวข้อมูล เพิ่ม -->
<td><?= $row['price'] * $row['stock'] ?></td>
```

2.2 ไฮไลต์สินค้าสต็อกน้อย

แก้บรรทัด `<tr>` ในรูป:

```
<tr style="<?= $row['stock'] < 30 ? 'background:#ffe0e0' : '' ?>">
```

2.3 ตรวจสอบราคาติดลบใน `create.php`

แก้ส่วนรับ POST:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name      = $_POST['name'];
    $category  = $_POST['category'];
    $price     = $_POST['price'];
    $stock     = $_POST['stock'];

    if ($price < 0) {
        $error = 'ราคาต้องไม่ติดลบ';
    } else {
        $sql = "INSERT INTO products (name, category, price, stock)
                VALUES ('$name', '$category', $price, $stock)";
        if (mysqli_query($conn, $sql)) {
            header('Location: index.php');
            exit;
        } else {
            $error = mysqli_error($conn);
        }
    }
}
```